

Nibli: Auditable Symbolic Reasoning with Bounded Evaluation and Retractable Premises

dhilipsiva

Abstract

Nibli integrates a formal predicate-call language, bounded symbolic evaluation, selective materialization, retractable premise identities, and optional evidence envelopes across native and WebAssembly execution. We describe its supported reasoning model and distinguish derivation from accepted premises from claims about input truth. Selected Lean proofs establish conditional model properties; implementation tests connect those models to code, while envelope validation checks structural coherence. A frozen research artifact compares finite Horn and stratified-negation workloads with clingo and compiled, single-threaded Soufflé, varies evaluation bounds, and exercises withdrawal and saved-state reopening. Across 1065 measured processes, no definitive-answer disagreement is observed, while 91 executions reach an external cutoff. The audit checks 1016 envelopes and 8036 source citations without a coherence or source-liveness error. Materialization recovers 24 returned definitive answers in the fixed bounds slice. At the 1,000-fact target, certificate query time is 39.9 times verdict-only query time by ratio of medians. Larger ingestion and forward recursion cases expose substantial limits under the cutoff. Native, Wasmtime, and Node/V8 agree on all 540 measured query outcomes in the shared scenarios. The results characterize an integration whose auditability depends on explicit premise identity, evaluation profiles, and evidence boundaries, while its performance and completeness remain workload-dependent.

1 Introduction

A technical decision can be logically justified and still be difficult to audit. Its premises may have changed, a negative result may depend on a closed-world assumption, and the returned explanation may describe a different execution profile from the one that produced the verdict. These problems become concrete when an analyzer supplies findings and a separate policy layer decides which ones permit a release. Removing a waiver should affect the decision and its evidence in a predictable way; an unfinished search should not masquerade as proof that no problem exists.

Nibli¹ is an implemented symbolic reasoning engine with a formal predicate-call surface, typed generated witnesses, bounded demand-driven evaluation, selective finite materialization, retained premise identities, and optional proof envelopes. It exposes operationally distinct definitive, unresolved, and resource-limited outcomes through native and WebAssembly execution paths. The system’s purpose is to make the relationship between accepted premises, evaluation choices, and evidence inspectable.

These ingredients have substantial precedents. Controlled languages connect restricted syntax to logic [3]; tabled logic programming manages reusable subgoals and completion [9]; Datalog systems provide high-performance evaluation and provenance [5, 11]. Formal models coupled to differential implementation tests are also an established verification approach [2]. We therefore do not propose a new general inference calculus or claim that auditability is unique to Nibli.

The contribution is a systems account and reproducible evaluation of their particular integration. First, we state the semantic and operational contracts, including the limits of closed-world answers, withdrawal, generated identity, and trusted computation. Second, we distinguish the Lean proof results from their implementation correspondence and from the weaker structural envelope check. Third, focused new experiments measure policy joins, recursive reachability, depth and materialization tradeoffs, evidence cost, premise

¹Source repository: <https://github.com/dhilipsiva/nibli>. Section 5 identifies the evaluated baseline and experimental source fingerprint.

transitions, and portable behavior. The protocol retains unfavorable workloads, non-definitive answers, and process cutoffs.

The research questions concern consistency under premise change, performance and completeness under evaluation choices, and behavior across execution surfaces. The resulting artifact includes the exact workloads, raw observations, proof envelopes, analysis scripts, and dependency locks. It supports a bounded claim about the evaluated engine and its evidence rather than a claim that symbolic reasoning certifies input truth or makes every logical problem tractable.

2 Reasoning model and operational contract

Nibli accepts a formal predicate-call language, nibli KR, and compiles it to an indexed first-order representation. Its executable semantics is a supported, bounded reasoning procedure over that representation, rather than an unrestricted first-order validity decision procedure. This distinction separates three objects: the statement a person intends, the formula accepted by the compiler, and the answer obtained by evaluating that formula against the current premises. Deterministic compilation addresses the second object. It does not establish agreement with the first.

2.1 A finite comparison fragment

We define the fragment used in the experiments independently of the implementation. Let C be the finite set of opaque constants occurring in a workload’s program and queries, and Σ a finite set of predicate symbols of fixed arity. A ground atom is $p(c_1, \dots, c_k)$ with $c_i \in C$. A program consists of a finite extensional set E and range-restricted rules

$$H \leftarrow A_1, \dots, A_m, \text{ not } B_1, \dots, \text{ not } B_n. \quad (1)$$

Every variable occurs in a positive body atom. There are no function symbols, existential heads, arithmetic operations, equality merges, or modal operators in this fragment. Extensional and intensional occurrences of the same predicate are permitted, as in the reachability experiment.

For each rule, put an edge from its head predicate to every body predicate, with negative polarity for a negated body atom. A stratification assigns a natural number $s(p)$ to each predicate such that $s(H) \geq s(A_i)$ and $s(H) > s(B_j)$. Nibli rejects registration that creates a strongly connected component containing a negative edge. A rejection is an input error, not an UNKNOWN answer. For accepted fragment programs, compute a least fixed point within each stratum after fixing the extensions of lower strata. We write the resulting perfect model as $M(E, P)$. The chain and policy workloads have hand-derived expected memberships in this model; clingo and Soufflé supply independently executed comparison results.

A negative body literal holds only when its positive atom is absent from the completed lower-stratum model. This is the closed-world assumption. It is not a claim that the physical world contains no further evidence. Similarly, queries quantify over the engine’s represented domain rather than every possible entity in the world. Range restriction means that a fresh query-only constant cannot create positive support merely by appearing in an adapter’s symbol universe. The comparison uses ground membership queries, not cross-system comparisons of quantified domains. Shuffling input order or consistently renaming those constants changes an execution workload without changing the expected abstract membership answers.

2.2 The broader KR surface

The implemented surface includes quantified arguments, connected clauses, equality, generated witnesses, tense, deontic wrappers, and opaque quoted abstractions. These features make the integrated engine more than a direct Datalog evaluator. They also prevent a claim that the entire surface has the same semantics as the comparison fragment. The artifact’s claim register separates implementation tests for these features from the narrower external solver comparison.

An ordinary multi-place predicate is represented through an event anchor and role predicates. Schematically,

Outcome	Meaning for a supported query in its recorded profile
TRUE	Evaluation finds a positive derivation, or an accepted trusted compute result establishes the query.
FALSE	Evaluation definitively fails to derive the claim in the current closed model, or a supported computation definitively refutes its supplied arguments. It is not, in general, a derivation of explicit negation.
UNKNOWN	Evaluation cannot settle the claim for a recorded reason, such as a cycle cut, unavailable compute backend, or non-finite computation.
RESOURCE_EXCEEDED	A configured depth or host resource limit prevents a definitive answer. The reason distinguishes depth, fuel, or memory.

Table 1: Operational result contract. An external experiment cutoff is recorded outside this table; it is not an engine result.

a ground surface assertion $p(a, b)$ introduces

$$\exists e. p(e) \wedge p_{x1}(e, a) \wedge p_{x2}(e, b). \quad (2)$$

The actual representation carries typed terms and shared node indices. Equation 2 is a readable projection, not a literal serialization of the intermediate buffer. Rules can introduce dependent event witnesses. Consequently, a function-free surface workload need not have a function-free internal encoding. Both materialization eligibility and external solver translation must account for this seam. Unspecified corpus places are also implicit in the KR compilation; the comparison concerns the explicitly supplied argument positions, used consistently in facts, rules, and queries.

Names, generated individuals, generated events, and compiler placeholders are different term cases. A generated witness has a semantic identity containing its source assertion ID, binder ordinal, sort, and origin; a dependent witness also depends on its arguments. A printed label such as `sk_0` is a display convenience. It is not an address that an input string can use to select the witness. This matters when a source is withdrawn or replayed: confusing a user constant with a generated witness would create joins and citations that have no supporting premise.

The represented domain includes named entities and admitted generated individuals, with finite numeric terms occurring in assertions also contributing members. Querying does not itself assert a new domain member. Equality can identify represented entities, and withdrawal reconstructs the domain and equality indexes. The default universal reading has no existential import: a universally stated restriction does not by itself establish a member of its restrictor class. An explicit legacy profile changes that behavior and is recorded in the proof envelope. All new measurements use existential import disabled.

Tense wrappers distinguish stored and derived flavors; they do not imply a clock, interval arithmetic, or automatic temporal lifting of bare rules. Opaque abstractions describe content without asserting that the content actually holds. These are representation commitments enforced by compilation and reasoning tests. They are not claims to cover arbitrary natural-language tense or modal logic. In particular, the finite comparison deliberately excludes those forms rather than silently flattening them into ordinary predicates.

2.3 Four outcomes, not four truth values

The public result type has four operational classes, with reasons on the two non-definitive classes (Table 1). Calling this an operational interface avoids assigning an epistemic probability or a many-valued model-theoretic meaning to a resource failure.

Negation must preserve this distinction. If proving A remains non-definitive, its absence has not been established, so a negation-as-failure check cannot turn that uncertainty into a definitive success. Connective combination also has decisive cases: a false conjunct settles a conjunction, while a true disjunct settles a disjunction. For unresolved combinations, resource exhaustion takes precedence over uncertainty. Reasons need not be commutative even when the result class is. These finite combinations are the part of the Lean-to-Rust correspondence that can be checked exhaustively.

Depth is an evaluation horizon, not a limit on the number of edges appearing in a dataset. A transitive rule can compose two already established subpaths. Materialization can also establish a complete finite extension that settles a query beyond the backward search horizon. Thus a chain longer than the configured depth need not return `RESOURCE_EXCEEDED`, and merely increasing the depth does not characterize elapsed time. The experiments measure definitive coverage alongside latency to expose this distinction.

2.4 Premise withdrawal and the trusted boundary

Let R be an ordered registry of accepted assertion records with stable IDs, and let $L(R)$ be its active subsequence. Ordinary withdrawal changes a record’s status and reconstructs logical state by replaying $L(R)$, preserving record identity. For a query q , the intended observational property is

$$\text{eval}(\text{withdraw}(R, i), q, \pi) \simeq \text{eval}(\text{replay}(L(R) \setminus \{i\}), q, \pi), \quad (3)$$

where π fixes the session profile. Verdict equivalence is the central observation; stable source IDs permit the evidence to be checked against the surviving registry as well. This property does not say that withdrawing one support removes every derivation. Alternative support must remain effective.

Some declarations intentionally survive rebuild, including restrictions on which predicates may be asserted directly. Equation 3 therefore applies to ordinary premises under the same admission state and configuration, not to treating every administrative declaration as a reversible logical fact. Reset and reopen are separate operations. Reopen reconstructs active logical records and retains withdrawn IDs; it is not a replay of a complete chronological audit log. The new update experiment explicitly supplies its operation sequence because retained status alone cannot recover that history.

The compute interface is another boundary of the argument. A backend evaluates supplied arguments and its reply is trusted for the current query. The reply is not automatically inserted into the durable premise registry. Failure to obtain a supported reply remains non-definitive. A computation step in a trace is evidence that the engine used that boundary, not a proof that the external implementation, input source, or freshness policy is correct. The current envelope does not supply a cryptographically authenticated backend transcript.

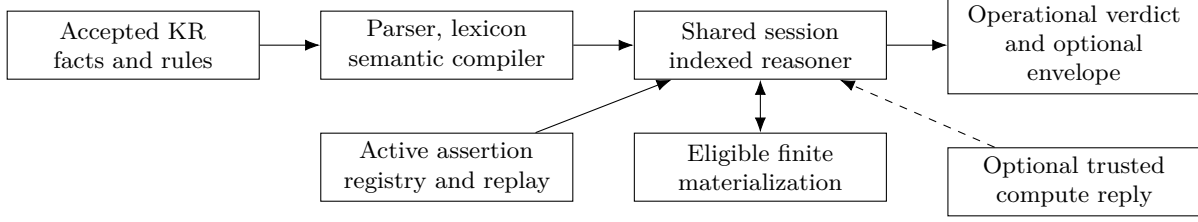
These qualifications give a precise interpretation of the project’s “zero hallucination” description: conclusions are constrained by accepted premises, supported inference, the recorded evaluation profile, and explicitly trusted computation. The phrase does not certify factual inputs, intended natural-language meaning, every implementation path, or unrestricted logical completeness. A wrong premise can yield a correctly derived wrong real-world decision.

3 Architecture and a technical adjudication case

The architecture places the same deterministic core behind several execution surfaces (Figure 1). The native embedding library, the WASI component, and the browser wrapper share parsing, semantic compilation, and reasoning crates through a common session layer. Runtime wrappers provide environmental functions and adapt the result representation. Shared code reduces the amount of independently maintained reasoning logic, but does not establish runtime equivalence: integer conversion, resource traps, serialization, and state recovery remain properties of the wrappers.

3.1 From statements to indexed evaluation

The front end accepts predicate names from a committed lexicon, with canonical argument places and converted aliases. The semantic compiler resolves that surface into typed, indexed buffers. Front-end acceptance and reasoning admission are different gates: a well-formed formula can still be refused because it violates stratification, an assertion restriction, or a query-only compute boundary. In the native persistent path, mutation stages candidate logical state, commits the authoritative registry, and then publishes the candidate. Precommit failure preserves the previous state; an ambiguous durable commit makes the live instance unavailable until a fresh open. This is a storage-state contract, not a logical inference rule.



Native Rust | WASI / Wasmtime | browser WASM / Node

Figure 1: The integrated pipeline and its trust boundaries. The measured solver comparison starts from a common finite workload description, before the three separate surface renderers. External analyzer and optional compute outputs remain trusted inputs, not independently established facts.

The fact store indexes relations and argument positions. Backward evaluation matches a query against stored facts and registered rule heads, builds substitutions, and recursively discharges bodies. Prepared inference work, candidate narrowing, and memoized completed work reduce repeated traversal. A depth-cut cache records the remaining budget associated with a safe cut. Cycle-contaminated results are not treated as reusable definitive failures: such a result depends on the current call path and would be unsound as a general absence certificate.

Selective materialization supplements this path. A negation-bearing query requests the relevant dependency cone before reasoning. Purely positive goals can remain on backward evaluation when it gives a definitive answer; a non-definitive result can request an eligible cone as a fallback. The requested roots accumulate within a knowledge-base state. The saturator projects eligible event decompositions back to finite surface atoms and computes consequences in stratum order using semi-naïve joins. A relation is marked complete only when its rules and dependencies meet the eligibility conditions.

Completeness is therefore an explicit property of a particular extension. Refusing a projection must not be confused with constructing an empty extension. Equality classes, compute conditions, unsupported flavors, or non-projectable term dependencies can prevent completion. If a dependency is refused, ordinary bounded reasoning remains responsible for its answers. This boundary is central to safe negative queries: an empty but incomplete set cannot justify negation-as-failure.

Updates interact with these caches. An insertion outside the requested dependency cone can leave its extensions valid. An eligible monotone insertion can resume from a delta. Growth that reaches a predicate used negatively requires recomputation because adding positive support can remove a negative consequence. Withdrawal follows reconstruction from surviving records, rebuilding domain, equality, and derived indexes together. The paper measures that implemented transactional cost, without treating the number of surviving records as a sufficient asymptotic model of all indexing and replay work.

3.2 Analyzer facts and reviewable policy

The running case adapts the repository’s technical adjudication example. An upstream analyzer supplies a flow, a sink classification, and optional sanitization evidence. A reviewer can supply an explicit waiver. Nibli evaluates the policy attached to those records. It does not calculate the program’s taint closure in this example and does not validate the analyzer’s accuracy.

The rules use `carries` for a flow, `dangerous` for a sink classification, `prevents` for a sanitizer, and `permits` for a waiver:

```

all $f, $d, $s, $o, $r:
  carries($f, $d, $s, $o, $r) &
  dangerous($s, $d, Exploit) & prevents(Sanitizer, $f)
-> authorized($f, Release, $s).

```

```

all $f, $d, $s, $o, $r:
  carries($f, $d, $s, $o, $r) &
  dangerous($s, $d, Exploit) & permits(Review, $f, Waiver)
  -> authorized($f, Release, $s).
all $f, $d, $s, $o, $r:
  carries($f, $d, $s, $o, $r) & ~authorized($f, Release, $s)
  -> warns(Gate, $f, $s).

```

The artifact stores each rule on one line for assertion. Formatting above only improves readability.

Authorization requires positive support. A flow whose sink was never classified is not cleared by absence of danger evidence; the finding rule still applies. This policy choice prevents one specific misuse of closed-world negation. It does not establish a universal policy that every missing fact should be treated conservatively: that decision belongs in the rules for the application.

The example also uses `admits` to close the direct-assertion vocabulary and `derived_only` to prevent direct assertion of authorization and findings. Those controls are tested in the existing adjudication gate. The shared performance fragment omits these administrative declarations so all systems receive equivalent ordinary rules and facts. This omission is disclosed; the benchmark does not compare authorization enforcement at each system’s input API.

3.3 Two supports, one withdrawal

For a particular flow, suppose both a sanitizer record and a waiver exist. The authorization query is true by either policy rule. Withdrawing the sanitizer must leave authorization true through the waiver. Withdrawing the remaining waiver makes authorization false and the finding true. Reasserting the sanitizer restores authorization and removes the finding. Reopening saved state must preserve the current outcome and the identity of surviving sources.

The update workload places a second flow alongside the first with only one sanitizer support. Its withdrawal must immediately remove authorization. These two paths discriminate alternative support from sole support: a test that only deletes the last support cannot show that the engine retained an independent one correctly. Queries after every transition also catch stale positive and negative caches; checking only the final reopened state could hide a temporary incorrect decision.

Evidence has its own obligations. Asserted leaves identify supporting registry records, and derived steps identify source rules. A fresh envelope after withdrawal must not cite a withdrawn record as active support, even when its top-level verdict happens to remain true. The artifact checks source liveness and exact source labels against the saved request and operation sequence. It retains the complete envelopes so this check can be repeated independently of the live engine.

3.4 Evidence generation is an execution choice

A verdict-only query can use a completed surface extension to answer a membership test. A trace needs more structure: which rule fired, which premises were used, and which witness connects an anchor to its roles. The current traced route therefore retains backward derivation work instead of replacing it with an unexplained materialized lookup. This creates a testable performance and completeness distinction between `query_holds` and `certify_text`.

An envelope binds the result and trace to the query, engine version, and effective profile, including depth and materialization settings. It is useful for auditing a particular execution, but is neither a signed record nor a self-contained semantic proof. It does not carry the entire premise registry or independently reconstruct the meaning of every rule. The source snapshots and update sequence in this artifact supply context that the envelope alone intentionally lacks.

Model	Established result	Implementation boundary
Combiner	Definitive connective outcomes have the specified decisive inputs; unresolved combinations preserve uncertainty/resource information.	Exhaustive comparison with the finite Rust result domain.
Stratification	Absence of a negative cycle is equivalent to existence of a valid stratification.	Predicate graphs constructed by the compiler and registration path require separate testing.
SCC	A correct SCC partition yields the same rejection criterion as mutual reachability.	The imperative Tarjan traversal is not verified; its output is compared with an independent reachability implementation.
Unification	Successful one-directional matching against a ground term produces a substitution instantiating the template to that term.	Groundness is a hypothesis; number payloads abstract away floating-point arithmetic. Rust enforces a ground stored-fact boundary.
Rule firing	Head matching plus satisfied premises of a model-sound rule entails the instantiated goal.	The rule’s validity in the model is a hypothesis; compilation and condition discharge need correspondence evidence.
Trace	Positive and negative certificates imply model membership and non-membership; compound query certificates inherit the result.	Assumes sourced facts, rule closure, candidate correctness, and support; not a complete verifier for serialized envelopes or every KR proof variant.

Table 2: Lean proof scope. File names and executable correspondence tests are listed in Appendix C and the artifact’s claim register.

4 Verification evidence and its scope

The assurance argument combines model-level proofs, implementation conformance tests, external solver comparisons, and state-transition tests. Each supports a different claim. Passing all of them is stronger evidence than any one alone, but their conjunction is not a machine-checked proof of the entire parser, compiler, runtime, and storage stack.

4.1 Mechanized models

The Lean files are self-contained formalizations checked by the locked Lean toolchain. Table 2 lists their principal results and the boundaries that matter for interpreting them. They model selected mechanisms; Rust code is not extracted from Lean.

The capstone is especially easy to overstate. Its `PerfectModel` structure contains an interpretation M , a base-fact predicate, finite candidate firings for each ground goal, and explicit assumptions. Base facts hold in M ; a candidate with satisfied positive premises and refuted negative premises concludes into M ; candidates identify the correct goal; and every true goal is either a fact or has a fireable supporting candidate. The file does not construct the perfect model from arbitrary KR source.

Positive certificates are depth-indexed: a goal is a base fact or a candidate has recursively certified premises. Negative certificates require that the goal is not a base fact and *every* candidate is blocked by a definitive premise. Induction gives

$$\text{Pos}_d(a) \Rightarrow M(a), \quad \text{Neg}_d(a) \Rightarrow \neg M(a).$$

Here d is the formal certificate index, not a measured Wasmtime fuel budget. The negative direction depends critically on candidate completeness and the support assumption. A trace omitting an alternative rule cannot establish closed-world absence merely because its displayed branch failed.

The implementation’s trace conformance test checks local step conditions and bridges asserted leaves to stored facts, derived steps to registered rules, and negative leaves to candidate-blocking evidence. Its corpus exercises the bridges and includes deliberately damaged evidence. It remains a corpus test. Specialized

existential, universal, count, compute, modal, and equality proof variants are not all covered by one end-to-end Lean theorem. Negated existential groups also have a narrower bridge than ordinary rule premises. We retain those limitations when citing the capstone.

4.2 Independent checks on the implementation

The native soundness gate uses Vampire for an applicable classical Horn fragment and clingo for stratified negation and closed-world behavior. The two oracles are necessary because treating default negation as classical negation would change the experiment’s semantics. Input filters and translation failures are part of the recorded evidence. A solver that was unavailable or never reached would establish nothing.

Metamorphic tests target operations for which a comparison of final answers alone is weak. Retraction is compared with replay of surviving premises, including quantified and numeric-domain queries. Materialization on and off must agree whenever both produce definitive answers; a non-definitive answer becoming definitive is counted separately. Registration tests compare negative cycle rejection against an independent graph criterion and query a fresh replay after rejection. These tests address stale state as well as isolated inference.

The KR seam gate tests compilation structure and metamorphic relationships between equivalent spellings. This is essential because testing a hand-built flat fact can miss the event/role decomposition actually produced by the front end. Persistence tests exercise retained IDs, withdrawn records, reopening, and commit-failure behavior. Runtime tests separately exercise component imports, fuel recovery, and native/Wasmtime/Node behavior. The artifact reruns these required checks with the relevant tools present and retains their output, rather than citing historical green CI.

4.3 What an envelope check establishes

The knowledge-base-independent envelope validator checks schema and profile constraints, step references, root/result coherence, and consistency of closed-world and negation-dependency flags. It does not establish that the claimed input was actually asserted, that every displayed rule application is semantically valid, or that every possible derivation has been considered. An internally coherent fabricated trace can therefore remain outside its assurance boundary.

The new artifact adds a distinct, limited check: every asserted or derived source citation must refer to a currently active record with exactly the recorded label. A deleted or mislabeled citation is a failure even if the envelope is coherent. Negative controls check that this audit rejects a withdrawn source, a changed label, and an empty source list. This improves the audit trail for premise changes without upgrading the envelope to an independent semantic proof checker. Substantive claims in the manuscript map to definitions, named Lean results, named tests, generated observations, or verified primary references.

5 Evaluation

The evaluation addresses three research questions: (RQ1) whether definitive answers and their evidence remain consistent with the declared semantics as premises change; (RQ2) what performance and completeness tradeoffs arise from bounded reasoning, materialization, and evidence generation; and (RQ3) which behaviors agree across native and WebAssembly execution. It evaluates the integrated engine at revision `158e3b0cdeeb`. The experimental source fingerprint is `618377685c101d2a`; the complete file hashes, dependency locks, commands, tool versions, and machine description are retained in the artifact.

5.1 Protocol and comparison boundary

Measurements run serially on the same local WSL2 machine using release builds. The host reports an AMD Ryzen 9 9950X3D 16-Core Processor, with 8 logical processors and 45.8 GiB of memory visible to WSL2. The artifact records CPU affinity, operating system, tool executable paths, and build profile. Rust uses optimization level 3, fat link-time optimization, one code-generation unit, and abort-on-panic. The locked paper shell includes clingo, Soufflé, Vampire, Lean, Wasmtime, Node, and the plotting and typesetting tools.

Verification and artifact compilation finish before performance collection; other activity on the workstation is an uncontrolled source of noise.

There are five fixed data seeds, one discarded warm-up per seed and case, and three measured repetitions. Workload and implementation order are randomized using a recorded seed. The policy family targets approximately 100, 1,000, and 10,000 asserted ground facts. The actual count depends on the seeded distribution of sanitizer and waiver records and is saved for each input. Each workload evaluates both a supported and an unsupported finding query. The chain family uses 4, 8, 12, 16, and 32 directed edges with one transitive rule. Its forward and reverse queries execute in separate fresh processes so a slow forward query cannot prevent observing the reverse result.

The bounds experiment crosses materialization on/off with depths 2, 5, 10, and 20 on a 100-fact policy workload and both directions of an eight-edge chain. This is a fixed diagnostic slice, not a full factorial crossing of all dataset sizes. The evidence experiment compares fresh verdict-only and certificate-generating sessions at each policy size. Update sequences use approximately 100 and 1,000 facts, with both kinds of support, reassertion, and two saved-state reopenings. The sizes and slices were frozen after harness validation and before collection.

For the cross-system comparison, separate renderers generate KR, clingo input, and Soufflé input from a common description of atoms and rules. They do not translate another engine’s emitted source. Hand-derived positive and negative cases validate argument order, variable sharing, default negation, and support alternatives. Predicate names are prefixed in the Soufflé adapter to avoid reserved built-ins; constants are represented as opaque symbols, not integers. The pilot exposed the reserved `cat` spelling and the final adapter explicitly tests that translation.

The comparison unit is a complete ground-query workload: process start, full input load, evaluation of every listed query, and output. A demand-driven engine may compute less of a relation than an eager engine while satisfying that same workload. Accordingly, these results are not a comparison of full relation enumeration throughput. Nibli loads one source statement per public `assert_text` call, retaining the resulting source IDs. Thus loading includes per-statement admission and state publication; this is not a bulk ingestion throughput measurement. The baselines load their generated input files through their ordinary command-line interfaces. clingo is invoked with one worker and a single stable model, sufficient for these stratified programs. Soufflé runs as a compiled executable with one worker, default optimizations, and no requested magic-set transformation. It reads external fact files, so compilation does not precompute an answer from embedded data. Its interpreted mode is not mixed into the comparison [8].

Preparation is recorded separately: neutral input translation, Soufflé source generation and C++ compilation, and executable reuse keyed by generated source. Nibli’s KR parsing and semantic compilation occur inside its loading phase. Building the generic Rust engine and installing the solver binaries are outside the workload timings for all systems. Compilation measurements cover 47 distinct programs, with a median of 9,224 ms and interquartile endpoints 9,069 and 9,403 ms. These costs should be included when choosing a system for a one-off job. Each distinct generated program is compiled once; these quartiles describe that set of preparations, not repeated builds of one program. Nibli-specific loading and query timings are useful phase diagnostics, but cannot be compared directly with another system’s whole-process latency.

5.2 Accounting for incomplete observations

Each execution has a 30-second external cutoff. A timeout is recorded as a process outcome, separately from `RESOURCE_EXCEEDED` returned by the engine. Standard output is flushed after each Nibli phase so completed loading and earlier answers remain visible when a later phase is cut off. Raw output, exit status, elapsed time, peak process resident memory, source snapshots, and update instructions accompany the observation.

Tables report medians and linear-interpolated quartiles over the 15 measured samples in a cell. Latency summaries use completed processes only; cutoff counts accompany them. They do not replace a censored sample with 30 seconds or silently omit it from the outcome ledger. Definitive-answer coverage uses all planned answers as the denominator, so a process that times out before answering does not improve coverage by disappearing. Repeated samples describe this machine and these seeded inputs, not independent deployment populations; we do not report significance tests or claim broad scaling laws.

Target facts	Load ms [Q1,Q3]	Withdrawals ms	Reopens ms	Completed
100	367.4 [351.4, 381.0]	7.80 [7.45, 9.03]	622.7 [598.2, 648.0]	15/15
1000	7,928 [7,620, 8,184]	309.4 [70.3, 425.1]	14,021 [13,649, 14,814]	14/15

Table 3: Native persistent update sequences. Reopen means closing and opening the database, with the profile reapplied. Timings are milliseconds; interval endpoints are the first and third quartiles.

Target facts	System	Wall ms [Q1, Q3]	RSS MiB	Cutoffs
100	clingo	4.98 [4.66, 5.42]	7.70	0/15
100	Nibli	47.0 [45.2, 48.7]	6.29	0/15
100	Soufflé	4.66 [4.37, 5.16]	5.65	0/15
1000	clingo	7.61 [7.17, 7.97]	7.95	0/15
1000	Nibli	3,950 [3,832, 4,057]	22.7	0/15
1000	Soufflé	5.28 [4.76, 5.60]	5.77	0/15
10000	clingo	30.3 [29.7, 31.0]	10.2	0/15
10000	Nibli	<i>no completion</i>	–	15/15
10000	Soufflé	8.14 [7.83, 8.55]	6.52	0/15

Table 4: Complete policy workloads, including process startup and input loading. RSS is peak resident memory for the whole process. The target size excludes rules; exact ground-fact counts are retained per seed.

5.3 Definitive answers and premise changes (RQ1)

Across the timed experiments, the ledger contains 1065 measured runs and 355 warm-ups. There are 0 disagreements between observed definitive answers and the workload expectations. Appendix D lists unresolved answers, resource outcomes, process failures, and cutoffs separately. These counts qualify the agreement claim: an absent answer is not evidence of semantic agreement.

The update sequence queries authorization and findings after every mutation. Within measured native evidence and update runs, the audit checks 1016 recorded certificate envelopes and 8036 source citations, with 0 source-liveness or label errors. These are emitted certificate and citation occurrences, not counts of independent proofs or unique premises. Table 3 separates loading, withdrawal, and reopen work. Withdrawal and reopening columns sum the operations of that kind within one sequence; certificate and reassertion phase measurements remain available in the CSV data.

This evidence supports the specific transition behavior of the integrated engine. The broader retraction differential supplies additional coverage for domain and equality reconstruction. Neither constitutes an incremental-update performance comparison with RDFox, clingo, or Soufflé; no equivalent baseline update API was measured.

5.4 Latency, bounds, and evidence (RQ2)

Table 4 and Figure 2 distinguish policy joins from recursive reachability. The repository documented a severe forward-query cost increase in an earlier measurement. Both directions and the unfavorable sizes remain in the frozen experiment. The new data determine how much of that behavior remains at this baseline; older timings are not copied into the results. In particular, an empirical increase over a short chain series is not by itself a proof of exponential asymptotic complexity.

Both comparators complete the measured policy workloads substantially faster than Nibli where all systems finish. Nibli’s loading phase dominates its 1,000-fact fresh-process cost, while the internal verdict queries take only a small part of that time. This result characterizes the per-statement public API workflow; it does not isolate the cost of each compiler, indexing, or state publication mechanism.

Materialization-on and materialization-off runs give 216 paired answers that are definitive in both modes, with 0 definitive disagreements. A further 24 paired answers change from a returned non-definitive outcome with materialization off to a definitive outcome with it on. External timeouts are not relabeled as such returned completeness gains. Figure 3 reports overall coverage for each mode and bound, which also exposes

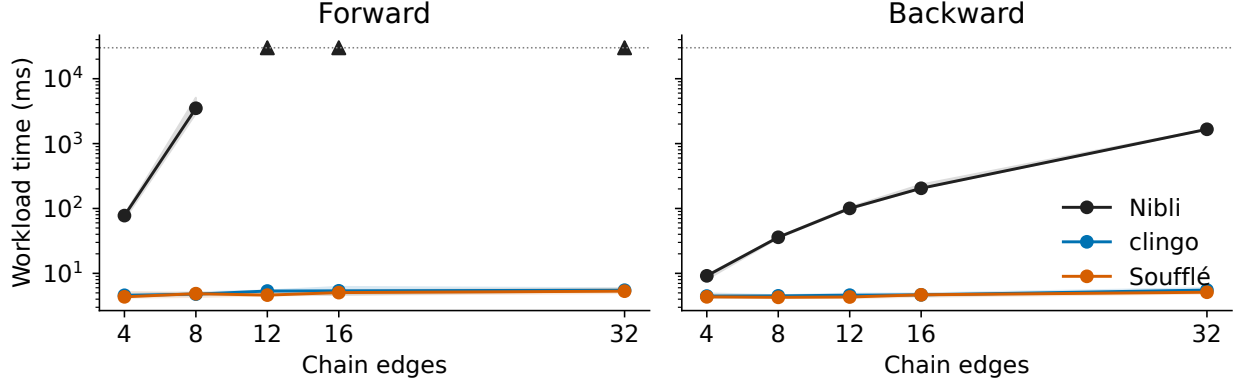


Figure 2: Fresh-process recursive reachability under the default comparison profile. Bands show interquartile ranges of completed runs. Triangles at the cutoff line indicate cells with externally timed-out runs; they are not completed latency points.

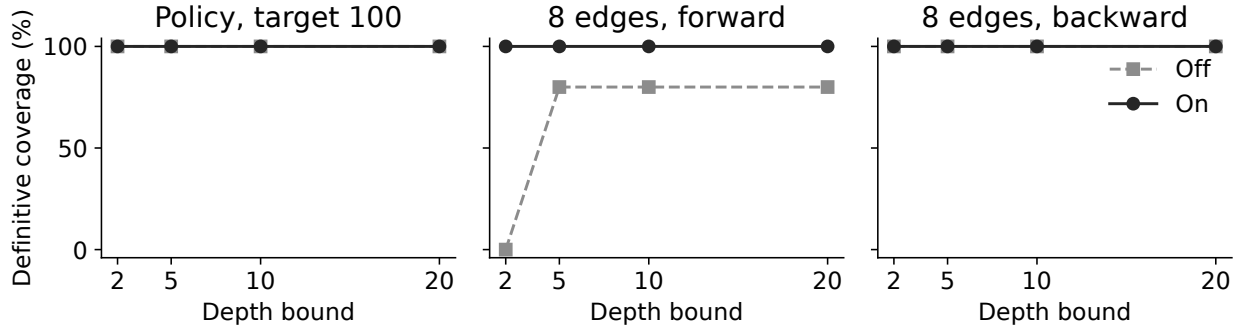


Figure 3: Definitive coverage in the fixed bounds slice. The denominator includes all planned answers, including those not emitted before a process cutoff.

missing answers. There are 0 paired changes in the opposite direction, from a returned definitive answer with materialization off to a non-definitive answer with it on. All observed gains are in the eight-edge forward workload. The unresolved off-mode answers at larger depths carry **CycleCut**: increasing the depth alone does not eliminate these call-path cuts, even though the input edge chain is acyclic. Policy and reverse-chain queries are already definitive throughout the tested bounds in both modes.

Table 5 measures the execution distinction described in Section 3. It compares fresh sessions so the verdict-only run cannot prewarm the traced run. The first and second queries within each session are ordered identically in both modes. This experiment measures an application choosing an evidence-producing API, not a constant serialization tax added to an otherwise identical inference path. The serialization phase times one encoding of the envelope. Constructing and writing the full raw log adds further harness work to whole-process time; it is outside the reported internal query and certificate timings.

The largest policy cases expose an ingestion limit in this workflow. Of 45 measured Nibli runs at the 10,000-fact target across the comparison and evidence groups, 45 reach the external cutoff; 45 do so before loading completes. These runs provide no completed query or certificate timing. The result concerns the recorded sequence of public API calls, including admission and state publication. It does not isolate parser throughput or prove that every bulk-ingestion strategy has the same limit.

The recursive asymmetry is also substantial: forward queries on chains of 12, 16, and 32 edges reach the cutoff in 45 of 45 measured runs, while 45 of 45 reverse-query runs complete. Dataset size alone therefore gives a poor account of the cost: the requested direction and evaluation path matter. A higher depth bound is not a time budget, and an eligible finite relation does not imply that the engine will eagerly complete it

Target facts	Verdict ms [Q1,Q3]	Certificate ms [Q1,Q3]	JSON bytes	Cutoffs V; C
100	0.345 [0.341, 0.362]	3.25 [3.13, 3.60]	105,191	0/15; 0/15
1000	1.99 [1.88, 2.10]	79.4 [77.9, 82.6]	971,521	0/15; 0/15
10000	<i>no completion</i>	<i>no completion</i>	–	15/15; 15/15

Table 5: Nibli evidence cost on fresh sessions. Each internal query measurement is the sum for the two policy queries. Certificate time includes reasoning and trace construction; validation and serialization are timed separately in the artifact. JSON size is the sum of emitted envelope bytes. Cutoffs are listed separately for verdict mode (V) and certificate mode (C).

before attempting a positive query.

At the 1,000-fact target, the ratio of median certificate time to median verdict-only query time is 39.9. This is a ratio of two fresh-session medians, not a paired speedup estimate. The median separately measured JSON serialization time is 0.707 ms. The distinction between evidence-producing reasoning and serialization is consequential: the former requests a trace through an execution path that retains backward derivation work. The comparison does not attribute every part of that difference to one internal optimization.

5.5 Portable behavior and host limits (RQ3)

The shared portability series covers policy findings, both reachability directions, withdrawal with alternative support, generated witness separation from an equal-looking user string, and tense discrimination. It uses the same request snapshots for native Rust, the WASI component executed by the Wasmtime host, and the wasm-bindgen wrapper under Node/V8. Five seeds, one warm-up, three recorded repetitions, and randomized runtime order yield 270 measured process scenarios and 540 query outcomes. There are 0 failed scenario comparisons. This tests verdicts and relevant profile fields, not byte-identical proof serialization or performance portability.

The comparison provides adequate host budgets. Fuel exhaustion and recovery are tested separately by the host suite: a deliberately small fuel budget produces a resource result, then raising the budget permits reconstruction, replay, and a subsequent definitive query. Browser execution does not expose the same fuel meter or durable database interface. Saved-state reopening is therefore established for the persistent native and Wasmtime paths by their respective checks, not attributed to the browser wrapper.

6 Related work

Nibli combines established mechanisms. The relevant comparison is therefore which contracts are integrated and measured, not whether symbolic inference, controlled syntax, provenance, or premise updates first appear here. Table 6 summarizes the precedents closest to the research questions. Its entries describe published capabilities, not outcomes of a new comparative usability or feature-completeness study.

Attempto Controlled English (ACE) gives a precisely constrained English surface and translates accepted text to discourse representation structures and other formal targets. Its surrounding tools include paraphrasing, reasoning, and rule execution [3]. Nibli’s predicate-call syntax pursues a more explicit argument-place discipline. That difference can be described structurally, but the present experiments do not establish that one surface is easier to author or less likely to be misunderstood. A formal parse can be determinate while a person still gives it an unintended reading.

Tabled logic programming supplies a long-established account of reusing subgoals, resuming suspended computations, and establishing completion. XSB also supports negation and incremental treatment of changing programs [9]. Nibli’s depth-sensitive caches should not be equated with a complete SLG evaluation strategy. Nor is combining top-down and bottom-up computation itself new: Warren describes their procedural integration through tabled evaluation [10]. The contribution here is the measured behavior of Nibli’s particular completion requests, bounded outcomes, state reconstruction, and evidence interface.

Soufflé synthesizes executable analyzers from Datalog [5]. Its provenance work augments evaluation with

Precedent	Established mechanism	Relationship to this study
Attempto Controlled English	Constrained language translated to formal representations, with paraphrasing and reasoning tools.	Precedent for determinate input interpretation; KR uses explicit predicate calls, and authoring quality is not measured here.
XSB / tabled logic programming	Reuse and completion of subgoal computations; tabled negation and support for changing programs.	Precedent for demand-driven evaluation; Nibli’s bounded path and selective completion have different operational limits.
Soufflé provenance	Annotations and on-demand proof-tree construction within a high-performance Datalog system.	Provenance is not unique to Nibli; this experiment measures Nibli envelopes and only verdict workloads across systems.
RDFox B/F maintenance	Incremental deletion handling that searches for alternative derivations.	Precedent for support-sensitive changes; Nibli’s measured withdrawal reconstructs surviving state.
Cedar	Mechanized executable models, differential random tests, and property-based implementation tests.	Precedent for combining model proofs with implementation evidence.
Logic-LM	Language-model formalization followed by symbolic solving and error-driven refinement.	Precedent for solver-assisted language workflows; this study starts with accepted formal premises.

Table 6: Related mechanisms. The accompanying discussion cites the corresponding primary publications and distinguishes them from the measured Nibli scope.

proof annotations and supports construction of partial proof trees for debugging [11]. It would therefore be incorrect to motivate Nibli by claiming that Datalog systems do not explain their answers. Our compiled Soufflé baseline measures ordinary evaluation with provenance disabled; it establishes no relative provenance overhead or certificate-strength result. Nibli’s envelope binds profile, outcome, and trace, while the structural checker remains weaker than an independent semantic proof verifier.

The Backward/Forward maintenance algorithm studied with RDFox seeks alternative derivations when facts are deleted and compares that work with Delete/Rederive [6]. This is directly relevant to the sole-support and alternative-support cases. Nibli instead reconstructs from retained assertions on withdrawal. Those designs address related correctness concerns with different update mechanisms. The new experiments do not measure RDFox updates or support a claim that reconstruction is generally faster.

Cedar’s verification-guided development combines proved executable models with differential random testing against production code and property-based testing of unmodeled components [2]. It is a methodological precedent for Nibli’s mixed assurance argument, not a baseline theorem prover. In both cases, the distinction between a proved model and evidence connecting production implementation to that model matters.

Logic-LM uses a language model to formulate a problem, a symbolic solver to reason over the formulation, and solver feedback for refinement [7]. This separates the quality of formalization from the reliability of solving an accepted formulation. Nibli has an optional formalization workflow, but no language model is in the measured reasoning loop. We make no comparative language-model accuracy claim, and solver success alone does not validate an intended natural-language meaning.

7 Limitations and threats to validity

The evaluation is intentionally local and finite. Synthetic policy records exercise a selected join pattern; a directed chain exercises one particularly unfavorable recursive rule. They do not represent the distribution of real authorization policies or static analyses. Larger, denser graphs, different selectivities, many rule alternatives, mixed query batches, and adversarial update schedules can produce different costs. The tested fact counts are input surface facts, not internal event-role tuples or full derived closures. No general scalability claim follows from their absolute latency. In particular, a timeout while loading through repeated single-statement API calls does not establish that every possible ingestion strategy fails at that dataset size. It does establish a cost of the measured public-API workflow.

Implementation order is randomized, but the experiment does not isolate the machine from its host operating system or control CPU frequency and competing workstation activity. Warm-ups reduce first-use effects without making the measurements warm in-process service timings. Peak RSS includes runtime and library overhead and cannot be read as the marginal size of a knowledge base. The 30-second cutoff also means that medians among completed samples can be optimistic for a partially censored cell. The outcome table, raw samples, and coverage plots preserve the information needed to notice that bias.

Cross-system equivalence is restricted to the declared finite fragment. The renderers are independent translations from common data, but they share that data generator and can share a misunderstanding of its intended semantics. Hand-derived discriminators and the existing independent solver gates reduce this risk without eliminating it. Nibli demand-driven queries and baseline materialization perform different amounts of internal work. The result is a comparison of delivering the requested answers, not an implementation-independent measure of inference steps. clingo also supports richer multi-shot workflows than the one-process invocation used here [4]; this experiment does not evaluate them.

Formal assurance remains partial. The Lean models do not prove compiler correctness, transaction implementation, all quantifier and modal behavior, materialization projection, or the complete WebAssembly toolchain. The trace theorem has explicit semantic hypotheses, and most correspondence tests are sampled. The source audit checks whether citations identify current premises, not whether each inference follows from them. Envelopes are not authenticated attestations. These limits preclude describing the system as a fully independently verified prover on every accepted input.

Closed-world policy quality is outside the experiment. A missing analyzer record, incorrect waiver, or incorrectly formalized policy can change an answer without exposing an engine defect. The example chooses positive authorization evidence, but an application must establish when its case records are complete enough to support a negative decision. Generated witnesses are logical representatives, not discovered real-world entities; separate witnesses do not by themselves prove that the represented individuals are unequal.

Finally, cross-runtime agreement covers the retained scenarios and supported profiles. It does not promise identical memory usage, latency, resource limits, or proof byte order. A synchronous external compute call is a host-side operation and is not automatically bounded by guest instruction fuel. External computation is excluded from the comparative workloads, and neither its correctness nor its real-world freshness is certified by this study.

8 Conclusion

Nibli makes several decisions visible that an ordinary Boolean query interface can conceal: whether a result is definitive, which premises support it, which closed-world assumptions it uses, and which evaluation profile produced it. The evaluated design combines bounded reasoning, selective completion, source-preserving withdrawal, and an evidence interface across shared native and WebAssembly code.

The experiments record no definitive disagreement or stale source citation, and the shared runtime scenarios agree. Those observations coexist with substantial limits: single-statement ingestion fails to finish at the largest policy target, forward recursion hits the cutoff on short chains, and certificate-producing queries cost more than verdict queries. Selective materialization improves returned definitiveness in the forward-chain slice. These results support the specific semantic, transition, and portability claims recorded in the artifact, together with their performance boundaries. The proof argument remains conditional at its model boundaries, and envelope coherence remains distinct from semantic verification. Future extensions that change materialization, witness handling, or persistence should preserve those distinctions and rerun the affected evidence, rather than inherit claims from an earlier revision.

Acknowledgment of AI assistance

OpenAI Codex provided substantial assistance with repository and literature analysis, experimental tooling, data analysis, and drafting and revising this manuscript. The measurements reported here were obtained by executing the recorded tools on the stated workloads. AI assistance does not supply an independent verification of the claims, and responsibility for the submitted work remains with the human author. This disclosure follows arXiv’s policy on significant use of generative language tools [1].

References

- [1] arXiv. Content moderation: Policy on the use of generative AI language tools. URL <https://info.arxiv.org/help/moderation/index.html>. Accessed 14 September 2026.
- [2] Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Mike Hicks, Kesha Hietala, John Kastner, Anwar Mamat, Matt McCutchen, Neha Rungta, Bhakti Shah, Emina Torlak, and Andrew Wells. How we built Cedar: A verification-guided approach. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024. doi: 10.1145/3663529.3663854. URL <https://www.amazon.science/publications/how-we-built-cedar-a-verification-guided-approach>.
- [3] Norbert E. Fuchs, Kaarel Kaljurand, and Tobias Kuhn. Attempto controlled english for knowledge representation. In *Reasoning Web*, volume 5224 of *Lecture Notes in Computer Science*, pages 104–124. Springer, 2008. doi: 10.1007/978-3-540-85658-0_3. URL https://attempto.ifi.uzh.ch/site/pubs/papers/reasoningweb2008_fuchs.pdf.
- [4] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Multi-shot ASP solving with clingo. *Theory and Practice of Logic Programming*, 19(1):27–82, 2019. URL <https://arxiv.org/abs/1705.09811>.
- [5] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. Soufflé: On synthesis of program analyzers. In *Computer Aided Verification (CAV)*, volume 9780 of *Lecture Notes in Computer Science*, pages 422–430. Springer, 2016. doi: 10.1007/978-3-319-41540-6_23. URL <https://souffle-lang.github.io/cav-paper>.
- [6] Boris Motik, Yavor Nenov, Robert Piro, and Ian Horrocks. Incremental update of datalog materialisation: the backward/forward algorithm. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015. doi: 10.1609/aaai.v29i1.9409. URL <https://www.cs.ox.ac.uk/people/ian.horrocks/Publications/download/2015/MNPH15b.pdf>.
- [7] Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3806–3824. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.248. URL <https://aclanthology.org/2023.findings-emnlp.248/>.
- [8] Soufflé Developers. Run Soufflé: Interpreter and compiler execution. URL <https://souffle-lang.github.io/execute>. Accessed 14 September 2026.
- [9] Terrance Swift and David S. Warren. XSB: Extending prolog with tabled logic programming. *Theory and Practice of Logic Programming*, 12(1–2):157–187, 2012. URL <https://arxiv.org/abs/1012.5123>.
- [10] David S. Warren. Top-down and bottom-up evaluation procedurally integrated, 2018. URL <https://arxiv.org/abs/1804.08443>. arXiv:1804.08443.
- [11] David Zhao, Pavle Subotić, and Bernhard Scholz. Debugging large-scale datalog: A scalable provenance evaluation strategy. *ACM Transactions on Programming Languages and Systems*, 42(2), 2020. doi: 10.1145/3379446. URL <https://souffle-lang.github.io/pdf/toplas20.pdf>.

A Reproduction protocol

The artifact is rooted at `paper/` in the engine checkout. The initial engine baseline is `158e3b0cdeeb`; the source fingerprint in `protocol.freeze.json` identifies the exact measured code and workload generator. The package contains an overlay for that checkout, including the private benchmark binary’s source and the locked environment. The source fingerprint is a content identity, not a claim that the working tree was committed under a new Git revision.

Enter the locked environment with:

```
nix develop --extra-experimental-features nix-command \
  --extra-experimental-features flakes .#paper
```

The recipe `just paper-check` runs required verification. Use `just paper-smoke` to validate adapters and `just paper-evaluate` to collect and analyze workloads. The recipe `just paper-build` regenerates the PDF, and `just paper-package` builds the source and data archives. The check recipe requires every relevant tool before invoking gates; missing tools cannot be reported as passing evidence. The README gives commands for writing a new result directory without overwriting the archived observations. Rebuilding the manuscript from the recorded results does not rerun experiments.

The preparation phase generates a neutral JSON description, KR assertions and queries, a clingo program, and a Soufflé program plus tab-separated fact files. Generated Soufflé source has a content hash and a separately recorded compilation. The schedule fixes seed, case, depth, materialization flag, mode, repetition, and implementation order. Every measured process receives a 30-second GNU `timeout`; a separate preparation cutoff applies to compilation. GNU `time` records peak RSS and CPU usage.

Observations are JSON Lines. A record identifies raw standard output, standard error, resource usage, status, answers, phase timings, and any definitive disagreement. The raw native output contains full envelopes before the summary removes them to avoid duplication. A completion manifest hashes the observation ledger. Analysis refuses an incomplete ledger or a changed hash. CSV exports contain measured samples, phase data, aggregate cells, and compilation data; PDF figures, LaTeX tables, and CSV exports are generated from the same aggregates.

The proof-source audit replays the saved operation sequence over assertion IDs and labels. A citation is accepted only when its record is active and its label matches exactly. This procedure has no rule interpreter and establishes no semantic proof independently of the engine. Its negative controls are part of the artifact tests.

B Hand-derived translation discriminator

The discriminator contains `dog(Ada)`, `dog(Bea)`, and `cat(Bea)`. The rule states that a dog without a cat fact is an animal. Thus `animal(Ada)` is true, `animal(Bea)` is false because its negative premise is blocked, and `animal(Cyd)` is false because its positive premise is absent. Both negative cases matter.

```
# KR
all $x: dog($x) & ~cat($x) -> animal($x).
dog(Ada). dog(Bea). cat(Bea).

% clingo
dog("Ada"). dog("Bea"). cat("Bea").
animal(X) :- dog(X), not cat(X).
answer(0) :- animal("Ada").
answer(1) :- animal("Bea").
answer(2) :- animal("Cyd").
#show answer/1.

// Souffle (facts supplied in p_dog.facts and p_cat.facts)
.decl p_dog(a0:symbol)
.decl p_cat(a0:symbol)
```

```

.decl p_animal(a0:symbol)
.input p_dog
.input p_cat
p_animal(X) :- p_dog(X), !p_cat(X).
.decl answer(i:number)
.output answer
answer(0) :- p_animal("Ada").
answer(1) :- p_animal("Bea").
answer(2) :- p_animal("Cyd").

```

The expected baseline output is the singleton answer index 0. The KR driver submits statements individually; the combined fact line above is typesetting shorthand. Other pilot cases cover two-step Horn inference, both reachability directions, waiver support, and a sink with no classification.

C Claim-to-evidence map

The machine-readable register `research/claims.json` expands the following mapping with file paths and symbols. Evidence classes are intentionally separate: a definition fixes terminology, a theorem proves a model property, a test exercises code, an observation records an execution, and a reference establishes a precedent.

Claim family	Primary evidence
Result combination	<code>proofs/Combiner.lean</code> ; <code>exhaustive_soundness_matches_lean_model</code> .
Negative-cycle criterion	<code>proofs/Stratification.lean</code> , <code>proofs/Scs.lean</code> ; <code>check_stratification_matches_proven_criterion</code> , <code>compute_sccs_matches_scc_spec</code> .
Ground matching and firing	<code>proofs/Unify.lean</code> , <code>proofs/RuleFiring.lean</code> ; <code>unify_conformance</code> , <code>rule_firing_conformance</code> .
Conditional certificate soundness	<code>proofs/Trace.lean</code> : <code>PerfectModel</code> , <code>cert_sound</code> , <code>qproof_sound</code> ; <code>trace_soundness_conformance</code> .
Envelope coherence	<code>nibli-types/src/logic.rs</code> : <code>validate_envelope</code> ; integration envelope matrix; new raw certificate records.
Withdrawal and retained identity	<code>nibli-verify/src/retract_diff.rs</code> ; engine/store persistence tests; new update sequences and citation audit.
Materialization agreement	<code>nibli-verify/src/materialize_diff.rs</code> ; new paired bounds observations and generated coverage figure.
Compiler seam and alias meaning	<code>nibli-verify/tests/nibli_kr_seam_gate.rs</code> ; surface compilation and alias conformance tests.
Cross-runtime behavior	Shared determinism corpus, host state/recovery suite, Node tests; <code>results/portability/</code> transcripts and requests.
Performance and incomplete execution	<code>results/evaluation/observations.jsonl</code> ; generated tables, sample CSV, process logs, and completion manifest.

D Complete outcome accounting and chain cells

Table 8 counts measured processes and emitted answer classes. The number of answers per process differs by group: updates query multiple stages, while a chain process contains one query. A cutoff can occur after earlier phases or answers were recorded. Therefore the answer columns must not be interpreted as a partition of the process columns. Unemitted answers are reflected in the coverage denominator.

Group	System	Runs	Cutoffs	Failures	Definite	U	R
comparison	Nibli	195	60	0	165	0	0
comparison	clingo	195	0	0	240	0	0
comparison	Soufflé	195	0	0	240	0	0
bounds	Nibli	360	0	0	456	9	15
evidence	Nibli	90	30	0	120	0	0
updates	Nibli	30	1	0	956	0	0

Table 8: Measured process and answer ledger. U denotes UNKNOWN; R denotes an engine RESOURCE_EXCEEDED result. Failures exclude external cutoffs.

Group	System	Family	Size	Last phase	Runs	RSS MiB
comparison	Nibli	chain	12	load	15	10.1
comparison	Nibli	chain	16	load	15	16.4
comparison	Nibli	chain	32	load	15	40.2
comparison	Nibli	policy	10000	open	15	55.6
evidence	Nibli	policy	10000	open	30	54.4
updates	Nibli	policy	1000	certificate	1	49.4

Table 9: External cutoffs grouped by the last completed phase, where available. An **open** row has no completed load observation. RSS is the median peak observed up to termination, not a completed workload’s memory requirement.

E Artifact and submission boundaries

The main text ends on page 14; references, this protocol, translation example, and expanded tables are supporting material. The upload archive contains the TeX sources, bibliography, generated `.bbl`, figures, and generated tables needed for offline typesetting. The separate research archive contains experimental sources, input snapshots, raw observations, proof envelopes, and reproduction instructions. Local DOCX chapters were consulted for exposition; their hashes and filenames are recorded, and their text and figures are not redistributed.

The proposed author is dhilipsiva, with no affiliation asserted, and the target arXiv category is cs.AI. No arXiv identifier, endorsement, acceptance, or submission is claimed. Significant AI assistance is disclosed with the references; submission and any endorsement communication are separate actions.

Edges	Direction	Nibli ms [Q1, Q3]	clingo ms	Soufflé ms
4	forward	77.6 [67.9, 80.4]	4.59 [4.38, 4.83]	4.35 [4.21, 5.19]
4	backward	9.16 [8.06, 9.44]	4.47 [4.21, 4.92]	4.35 [4.19, 4.72]
8	forward	3,504 [2,910, 5,104]	4.78 [4.60, 5.10]	4.86 [4.32, 5.00]
8	backward	36.0 [35.0, 36.6]	4.50 [4.33, 4.61]	4.27 [4.16, 4.52]
12	forward	<i>no completion</i> [†]	5.34 [4.78, 5.45]	4.62 [4.55, 4.92]
12	backward	100.3 [97.8, 104.5]	4.61 [4.40, 4.87]	4.32 [4.16, 4.52]
16	forward	<i>no completion</i> [†]	5.38 [4.67, 6.19]	5.05 [4.67, 5.47]
16	backward	204.7 [202.0, 234.2]	4.67 [4.49, 4.91]	4.67 [4.36, 4.83]
32	forward	<i>no completion</i> [†]	5.50 [5.37, 5.89]	5.31 [4.97, 5.81]
32	backward	1,661 [1,644, 1,666]	5.53 [5.21, 5.93]	5.12 [4.89, 5.25]

Table 10: Every chain cell. Values are milliseconds with first and third quartiles. A dagger marks any externally timed-out sample in the cell. Exact cutoff counts and individual samples are in `generated/summary.csv`.